CSE333 LAB

Signals

Zuhal Altuntaş

zuhal.altuntas@marmara.edu.tr

[Adopted from UNIX SYSTEMS Programming: Communication, Concurrency, and Threads]

Basic Signal Concepts

- A signal is a software notification to a process of an event

Basic Signal Concepts

- A signal is a software notification to a process of an event
- A signal is **generated** when the event that causes the signal occurs
- A signal is **delivered** when the process takes action based on the signal
-

Basic Signal Concepts

- A signal is a software notification to a process of an event
- A signal is **generated** when the event that causes the signal occurs
- A signal is **delivered** when the process takes action based on the signal
- The **lifetime** of a signal is the interval between its generation and its delivery
- A signal that has been generated but not yet delivered is said to be **pending**
  - There may be considerable time between signal generation and signal delivery
- The process must be running on a computer at the time of signal delivery

Basic Signal Concepts

- A signal is a software notification to a process of an event
- A signal is **generated** when the event that causes the signal occurs
- A signal is **delivered** when the process takes action based on the signal
- The **lifetime** of a signal is the interval between its generation and its delivery
- A signal that has been generated but not yet delivered is said to be **pending**
  - There may be considerable time between signal generation and signal delivery
- The process must be running on a computer at the time of signal delivery
- A process **catches** a signal if it executes a **signal handler** when the signal is delivered
- A program **installs** a signal handler by calling the `sigaction()` function with the name of a user-written function
- If the process is set to **ignore** a signal, that signal is thrown away when delivered and has no effect on the process

Basic Signal Concepts (continued)

- The action taken when a signal is generated depends on the current signal handler for that signal and on the process signal mask
- The signal mask contains a list of currently blocked signals
- Blocked signals are not thrown away as ignored signals are
- If a signal is blocked, it is delivered when the process unblocks that signal
- A program blocks a signal by changing its process signal mask using `sigprocmask()`
- A program ignores a signal by setting the signal handler to SIG_IGN using `sigaction()`

Certain POSIX Signals

SIGNAL	NBR	DESCRIPTION	DEFAULT ACTION
SIGABRT	6	Process abort (sent by assert)	Implementation dependent
SIGALRM	14	alarm clock	Abnormal termination
SIGCONT	25	Execution continued if stopped	Continue
SIGFPE	8	Arithmetic error (i.e., divide by zero)	Implementation dependent
SIGINT	2	Interactive attention signal (Ctrl-C)	Abnormal termination
SIGKILL	9	Termination (cannot be caught or ignored)	Abnormal termination
SIGQUIT	3	Interactive termination; core dump	Implementation dependent
SIGSEGV	11	Segmentation fault	Abnormal termination
SIGSTOP	23	Execution stop (cannot be caught or ignored)	Stop
SIGTERM	15	Termination	Abnormal termination
SIGUSR1	16	User-defined signal 1	Abnormal termination
SIGUSR2	17	User-defined signal 2	Abnormal termination

kill Command

- A user can send signals to a process from a command shell using the `kill` command
- The `kill` command sends a signal to the process or processes that are specified by each `pid` operand

```
kill [-signal_name] pid ...
kill [-signal_number] pid ...
```

- The first form of the `kill` command take a signal name as defined in `signal.h` without the SIG prefix (i.e., INT instead of SIGINT)

- The second form takes a signal number
- If no signal name or number is specified, SIGTERM is sent by default

- Example usage

```
kill 3425
kill –9 896
kill –KILL 3425
kill –INT 9328
kill –USR1 567

kill -l      :will list the signals the system understands
```

kill() Function

In a program, a user can send a signal to a process using the `kill()` function

```
#include <signal.h>

int kill(pid_t pid, int signalNbr);
```

- The function takes a process ID and a signal number (i.e., a symbolic name) as parameters
- If successful, the function returns zero; otherwise, it returns –1 and sets `errno`
- A user may only send a signal to a process that he or she owns
- Example use of the `kill()` function

```
int status;
pid_t childPid;
childPid = fork();
. . .

if(childPid > 0){
     status = kill(childPid, SIGUSR1);
     if (status == -1)
          perror("Failed to send the SIGUSR1 signal to child");
 }
```

Signal Mask and Signal Sets

- #include <signal.h>
- int sigemptyset(sigset_t *set);
  - sigemptyset initializes the set to contain no signals
- int sigfillset(sigset_t *set);
  - sigfillset puts all signals in the set
- int sigaddset(sigset_t *set, int signo);
  - sigaddset adds one signal to the set
- int sigdelset(sigset_t *set, int signo);
  - sigdelset removes one signal from the set
- int sigismember(const sigset_t *set, int signo);
  - sigismember tests to see if a signal is in the set
  -

Ex:

```
  If ((sigemptyset(&twosigs) == -1) || (sigaddset(&twosigs, SIGINT) == -1) ||
(sigaddset(&twosigs, SIGQUIT) == -1))

    perror("Failed to set up signal mask");
```

Process Signal Mask

- #include <signal.h>
- int sigprocmask(int how, const sigset_t *restrict set, sigset_t *restrict oset);
- The how can be:
  - SIG_BLOCK
  - SIG_UNBLOCK
  - SIG_SETMASK
- The process signal mask is a set of signals that are blocked.
- A blocked signal remains pending after it is generated until the signal is unblocked.
- The process signal mask is inherited by the child process.
- Ex:
- ```
  sigset_t newsigset;

   if ((sigemptyset(&newsigset) == -1) || (sigaddset(&newsigset, SIGINT) == -1))
  perror("Failed to initialize the signal set");

   else if (sigprocmask(SIG_BLOCK, &newsigset, NULL) == -1)
  perror("Failed to block SIGINT");
  ```
  ```

Catching and Ignoring Signals: sigaction

- #include <signal.h>
- int sigaction(int signo, const struct sigaction *act, struct sigaction *oact);

- struct sigaction {

```
    void (*sa_handler)(int);  // SIG_DFL, SIG_IGN or pointer to function
    sigset_t sa_mask;         // additional signals to be blocked during execution of handler
    int sa_flags;             // special flags and options
    void(*sa_sigaction) (int, siginfo_t *, void *);   // realtime handler
};
```

sigaction() Function

The `sigaction()` function allows the calling program to set and examine the action associated with a specific signal

```
#include <signal.h>

int sigaction(int signalNbr, const struct sigaction *action, struct sigaction
*oldAction);
```

-
  - The `signalNbr` parameter specifies the signal number for the action
  - The `action` parameter is a pointer to a `struct sigaction` structure that specifies the action to be taken
  - If `action` is NULL, the call to the function does not change the action associated with the signal
  - The `oldAction` parameter is a pointer to a `struct sigaction` structure that is assigned the previous action associated with the signal
- If successful, the function returns zero; otherwise, it returns –1 and sets `errno`

struct sigaction

The `struct sigaction` structure has the following contents:

```
struct sigaction
{
    void (*sa_handler)(int); // SIG_DFL, SIG_IGN or pointer to function
    sigset_t sa_mask;        // additional signals to be blocked during execution of handler
    int sa_flags;            // special flags and options
    void (*sa_sigaction)(int, siginfo_t *, void *); // realtime handler
};
```

The storage for `sa_handler` and `sa_sigaction` may overlap; consequently, a program should use only one of these members to specify the action

-
- Ex:
-
```
struct sigaction newact;
newact.sa_handler = mysighand; /* set the new handler */
newact.sa_flags = 0;       /* no special options */
if ((sigemptyset(&newact.sa_mask) == -1) || (sigaction(SIGINT, &newact, NULL) == -1))
       perror("Failed to install SIGINT signal handler");
```

Example SIGINT Signal Handler

```
#include <signal.h>
#include <stdio.h>
#include <unistd.h>
#define MAX_PRESSES 5

static int pressCount = 0;

void catchCtrlC(int signalNbr);

int main(void)
{
// See next slide
}

// This function is the signal handler
void catchCtrlC(int signalNbr)
{
char message[] = "Ctrl-C was pressed\n";

write(STDERR_FILENO, message, strlen(message) );
pressCount++; // Global variable
} // End catchCtrlC
```

The `catchCtrlC()` function is defined here as a signal handler for the SIGINT signal generated by Ctrl-C.

The `write()` function is used instead of `fprintf()` because POSIX guarantees that it is async-signal safe, meaning that the function can be called safely from within a signal handler.

(More on next slide)

Example SIGINT Signal Handler (continued)

```
// ****************************************
int main(void)
{
struct sigaction action;
int status;

action.sa_handler = catchCtrlC;
action.sa_flags = 0;
status = sigemptyset(&action.sa_mask);
if (status == -1)
   {
   perror("Failed to initialize signal set");
   exit(1);
   } // End if
status = sigaction(SIGINT, &action, NULL);
if (status == -1)
   {
   perror("Failed to set signal handler for SIGINT");
   exit(1);
   } // End if

while (pressCount < MAX_PRESSES); // Loop has no statements
return 0;
} // End main
```

Sample Output

```
%$ a.out
Ctrl-C was pressed
Ctrl-C was pressed
Ctrl-C was pressed
Ctrl-C was pressed
Ctrl-C was pressed

%$
```

Example (ch08/signalterminate.c)

```
#include <math.h>
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
static volatile sig_atomic_t doneflag = 0;
/* ARGSUSED */
static void setdoneflag(int signo) {
   doneflag = 1;
}
int main (void) {
   struct sigaction act;
   int count = 0;
   double sum = 0;
   double x;

   act.sa_handler = setdoneflag;   /* set up signal handler */
   act.sa_flags = 0;
   if ((sigemptyset(&act.sa_mask) == -1) ||
       (sigaction(SIGINT, &act, NULL) == -1)) {
      perror("Failed to set SIGINT handler");
      return 1;
   }

   while (!doneflag) {
      x = (rand() + 0.5)/(RAND_MAX + 1.0);
      sum += sin(x);
      count++;
      printf("Count is %d and average is %f\n", count, sum/count);
   }

   printf("Program terminating ...\n");
   if (count == 0)
      printf("No values calculated yet\n");
   else
      printf("Count is %d and average is %f\n", count, sum/count);
   return 0;
}
```

How to Compile:
gcc signalterminate.c –o signalterminate.out -lm

How to Run:
./signalterminate.out